

GRAFIKA KOMPUTEROWA I KOMUNIKACJA Z KOMPUTEREM

Wykład 2

Andrzej Śluzek

pokój: 3/64

email: andrzej_sluzek@sggw.edu.pl



Tematyka wykładów i zajęć laboratoryjnych

(oficjalny sylabus, kolejność i tematyka może być nieco inna)

1. Wstęp. Urządzenia graficzne wejścia-wyjścia.
2. Podstawowe pojęcia i definicje dotyczące obrazów cyfrowych (w technice rastrowej i wektorowej) oraz modeli kolorów.
3. Punktowe, lokalne i globalne metody przekształcania obrazów.
4. Transformacje geometryczne obiektów graficznych i wypełnianie figur.
5. Eliminacja elementów zasłoniętych, metody kreślenia krzywych.
6. Modele oświetlenia i cieniowania.
7. Przekształcenia morfologiczne i ich zastosowania.
8. Transformaty obrazów i sygnałów dźwiękowych (FFT, Hough, Radon). Wybrane metody filtracji danych jedno- i dwuwymiarowych.
9. Wybrane modele interakcji człowiek-komputer. Zasady projektowania i analizy interfejsów komputerowych.
10. Interfejsy dla osób niepełnosprawnych. Interakcja człowiek-komputer dla komputerów przyszłości.

Przetwarzanie (przekształcanie) obrazów cyfrowych

Przetwarzanie obrazów jest operacją wykorzystywaną zarówno przy pobieraniu obrazów (np. z kamery) jak i przy generowaniu obrazów (np. przez grafikę komputerową).

Istotą przetwarzania obrazów jest, że zarówno dane wejściowe jak i wyjściowe mają postać obrazu.

Pewne szczególne aspekty przetwarzania obrazów omawiane były na Wykładzie 1. Np. zmiana modelu obrazu z RGB na HSV jest przetworzeniem polegającym na zamianie 3-wymiarowych wektorów (kolorów pixeli) w 3-wymiarowe wektory w innej przestrzeni.

W najbardziej ogólnym sensie, istnieją trzy grupy metod przetwarzania obrazów:

- Metody punktowe
- Metody lokalne
- Metody globalne

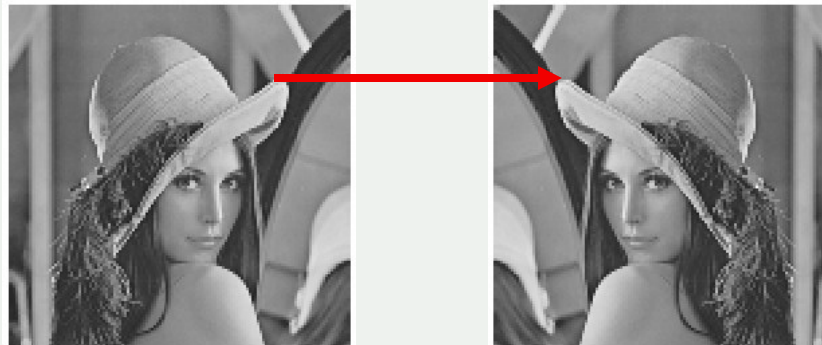
Punktowe metody przetwarzania obrazów

Istotą metod punktowych jest to, że:

- Jasności/kolory pojedynczego pixela obrazu przetworzonego (wyjściowego) są funkcją jasności/kolorów pojedynczego pixela obrazu oryginalnego (wejściowego).

$$i_{wy}(m,n) \underset{f}{\Leftarrow} i_{we}(k,l)$$

Najczęściej współrzędne wyjściowe są takie same jak wejściowe (tzn. $(m,n) = (k,l)$), ale nie zawsze tak jest, np. zwierciadlane odbicie obrazu:



Np. dla obrazów 256×256

$$i_{wy}(m,n) = i_{we}(257 - m, n)$$

albo

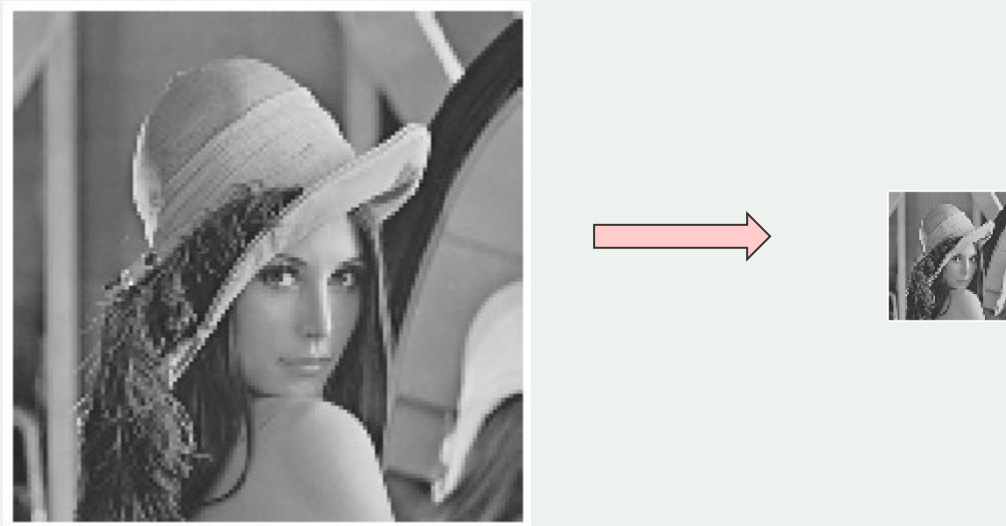
$$i_{wy}(m,n) = i_{we}(255 - m, n)$$

W tym przypadku funkcja f jest po prostu identycznością.

Punktowe metody przetwarzania obrazów

Inny przykład to próbkowanie (zmniejszanie rozdzielczości w najprostszy sposób) obrazu, np.

$$i_{wy}(m, n) = i_{we}(4m, 4n)$$



W tym przypadku funkcja f jest również identycznością.

Punktowe metody przetwarzania obrazów

W większości typowych metod punktowych, współrzędne pozostają jednak takie same, tzn.

$$i_{wy}(m,n) \stackrel{f}{\Leftarrow} i_{we}(m,n) \quad \text{czyli} \quad i_{wy}(m,n) = f[i_{we}(m,n)]$$

Przykładowe punktowe operacje przetwarzania obrazów:

1. Zmiana formatu obrazu (np. z RGB do HSV). Przykłady omawiane poprzednio.

2. Zmniejszenie wymiarowości obrazu (np. kolorowy na monochromatyczny). Funkcja f może być różnie definiowana.

$$f(R,G,B) = \frac{R+G+B}{3} \quad \text{intuicyjna zamiana}$$

$$f(R,G,B) = 0.299R + 0.587G + 0.114B \quad \text{wykorzystanie kanału Y formatu YCbCr}$$

$$f(R,G,B) = 0.2126R + 0.7152G + 0.0722B \quad \text{wykorzystanie kanału Y formatu YUV}$$

Punktowe metody przetwarzania obrazów

Przykład:



$$f(R, G, B) = \frac{R + G + B}{3}$$

$$f(R, G, B) = 0.299R + 0.587G + 0.114B$$

$$f(R, G, B) = 0.2126R + 0.7152G + 0.0722B$$

3. Zwiększenie wymiarowości obrazu (np. monochromatyczny na kolorowy).

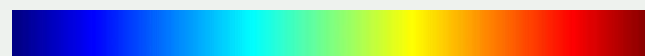
Operacja ta zwana jest **pseudo-kolorowaniem** obrazu. Popularna zwłaszcza przy wizualizacji obrazów medycznych (np. rentgenowskich, tomograficznych, czy z rezonansu magnetycznego).

Istnieje wiele popularnych wersji pseudo-kolorowania (np. *sine*, *hot*, *warm*, *jet*) z odpowiednio zdefiniowanymi funkcjami (mapami kolorów) f .



hot

warm



jet



sine

Punktowe metody przetwarzania obrazów

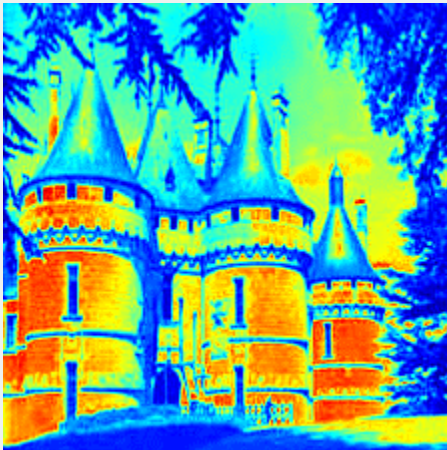
Np. funkcja f dla pseudo-kolorowania *sine* to:

$$\begin{cases} r(i) = -0.5 \cos(\pi i) + 0.5 \\ g(i) = \sin(\pi i) \\ b(i) = 0.5 \cos(\pi i) + 0.5 \end{cases}$$

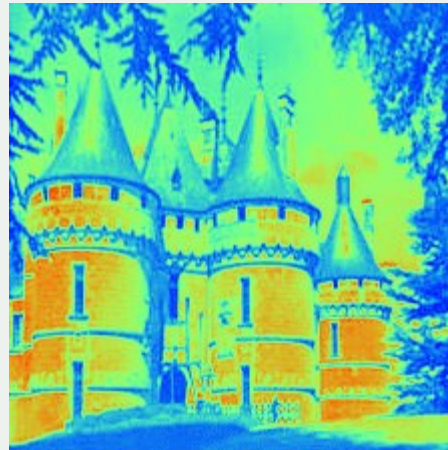
Przykład:



hot



jet



sine



warm

Punktowe metody przetwarzania obrazów

4. Modyfikacja jasności lub kolorów obrazu.

Istnieje wiele różnych takich operacji, a najważniejsze z nich to:

- a. Obrazy odwrotne (negatyw).
- b. Binaryzacja (progowanie) obrazu.
- c. Korekcja jasności (kolorów), np.
 - Gamma-korekcja
 - Wyrównywanie (normalizacja) histogramu

(a) Przykłady negatywów (mono- i polichromatyczne):



Punktowe metody przetwarzania obrazów

4. Modyfikacja jasności (kolorów) obrazu.

b. Binaryzacja (progowanie) obrazu.

Najprostsza metoda to „ręczny” wybór progu

$$i_{bin}(m,n) = \begin{cases} 1(\text{lub } 255) & \text{if } i(m,n) \geq T \\ 0 & \text{if } i(m,n) < T \end{cases}$$

Czy istnieje „najlepszy” próg? Można go wyznaczyć stosując tzw. **metodę Otsu**.



$T = 50$



$T = 128$



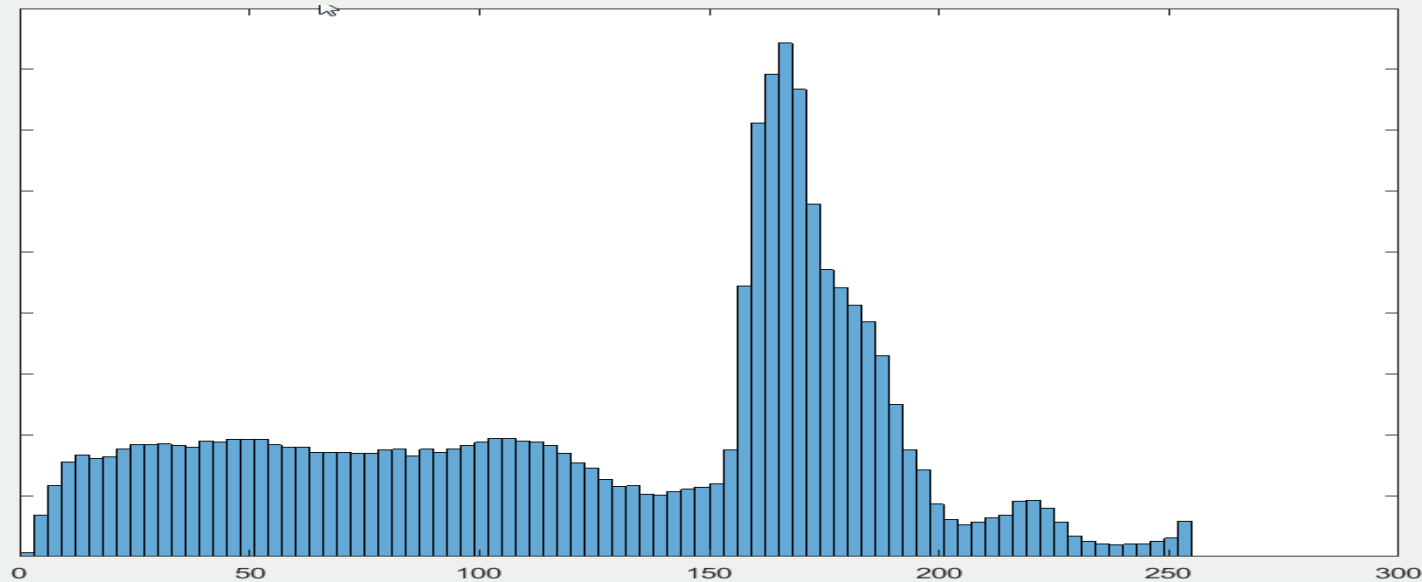
$T = 200$

Punktowe metody przetwarzania obrazów

4. Modyfikacja jasności (kolorów) obrazu.

W **metodzie Otsu** wybieramy próg, który minimalizuje zróżnicowanie wśród pixeli uznanych za białe, oraz wśród pixeli uznanych za czarne.

Do znalezienia progu wg metody Otsu konieczne jest wyznaczenie *histogramu jasności* w obrazie..



Punktowe metody przetwarzania obrazów

4. Modyfikacja jasności (kolorów) obrazu.

Następnie wybieramy taki próg, przy którym jest najmniejsze odchylenie standardowe jasności wśród pixeli sklasyfikowanych jako *białe* oraz najmniejsze odchylenie standardowe jasności wśród *czarnych* pixeli.

$$\sigma_{wh} = \sqrt{\frac{1}{k_{wh}} \sum_{i(m,n) > Th} (i(m,n) - \overline{i_{wh}})^2} \quad \text{gdzie } k_{wh} \text{ to liczba pixeli takich, że } i(m,n) > T,$$

a $\overline{i_{wh}}$ to średnia jasność takich pixeli.

$$\sigma_{bl} = \sqrt{\frac{1}{k_{bl}} \sum_{i(m,n) \leq Th} (i(m,n) - \overline{i_{bl}})^2} \quad \text{gdzie } k_{bl} \text{ to liczba pixeli takich, że } i(m,n) \leq T,$$

a $\overline{i_{bl}}$ to średnia jasność takich pixeli.

W tym przykładzie taki optymalny próg wynosi 115.



Otsu ($T = 115$)

Punktowe metody przetwarzania obrazów

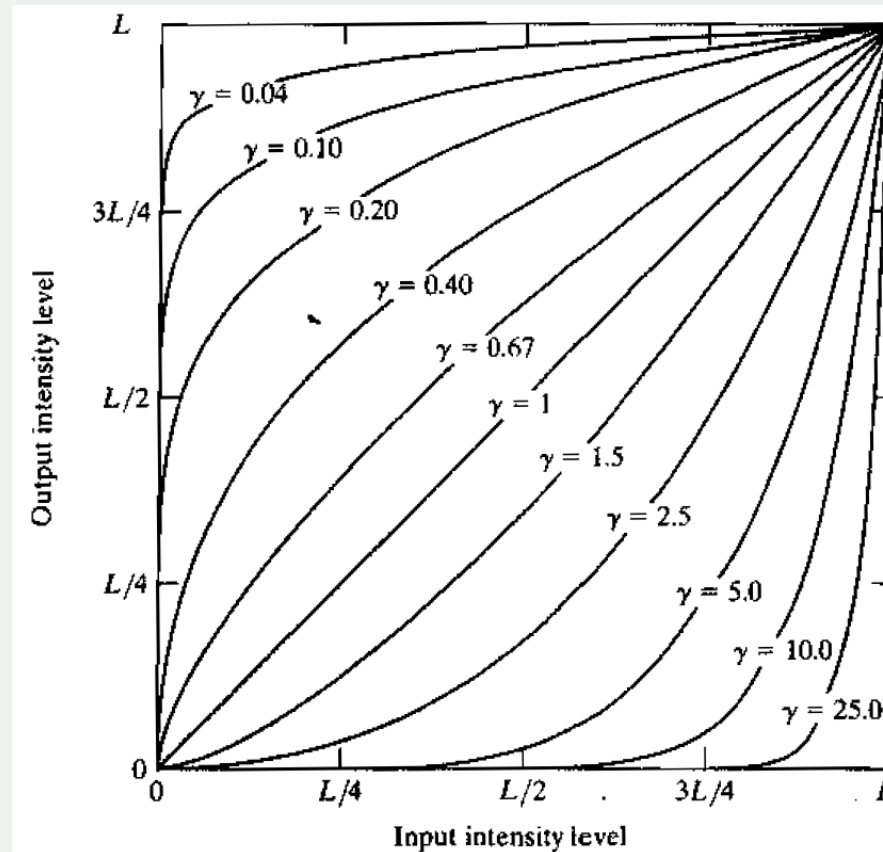
4. Modyfikacja jasności (kolorów) obrazu.

c. Korekcja jasności (kolorów)

➤ Gamma korekcja

Oryginalne jasności (zakładamy, że są znormalizowane do zakresu $[0;1]$) zmieniamy wg wzoru:

$$i_{new} = i_{old}^{\gamma}$$



Punktowe metody przetwarzania obrazów

➤ Gamma korekcja



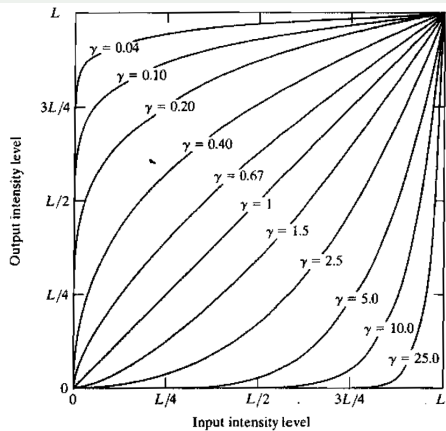
$$\gamma = 1$$



$$\gamma = 10$$

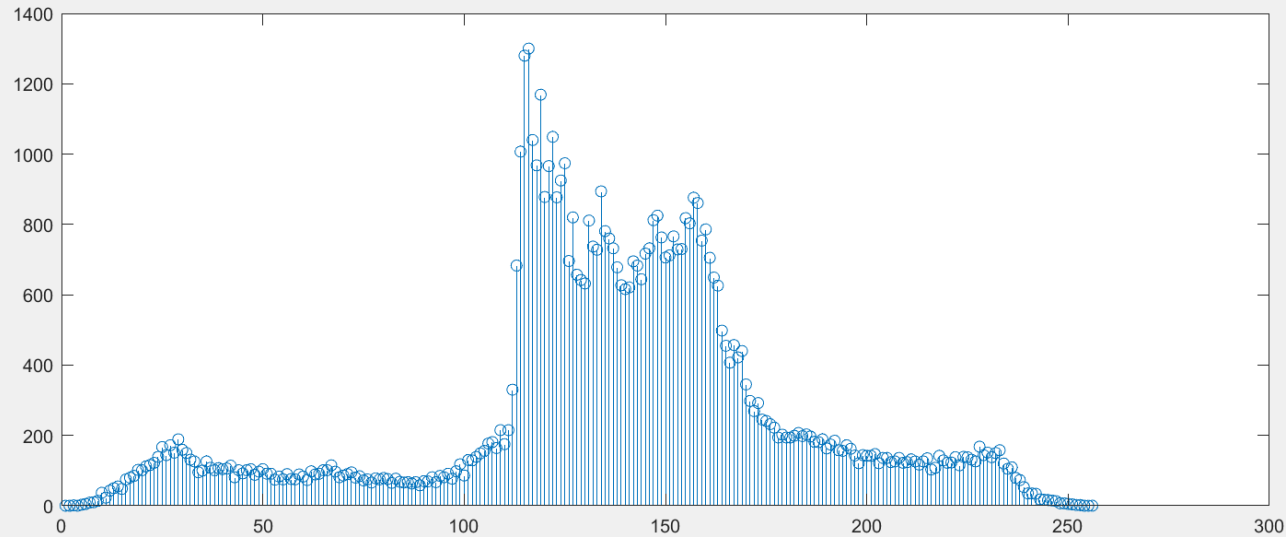


$$\gamma = 0.25$$



Punktowe metody przetwarzania obrazów

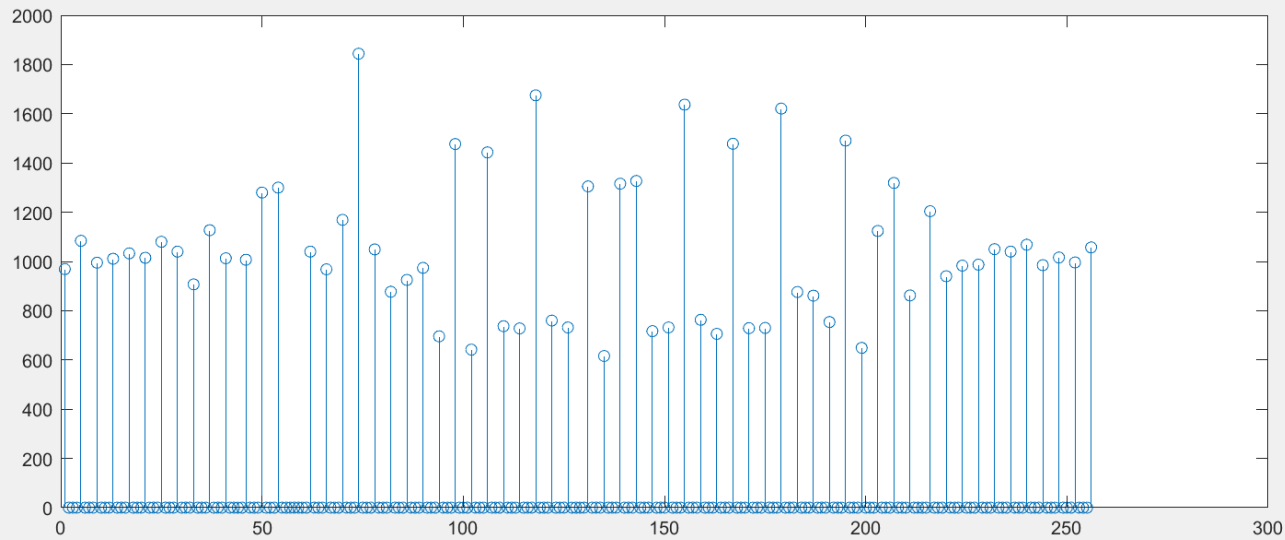
➤ Wyrównywanie (normalizacja) histogramu



Punktowe metody przetwarzania obrazów

➤ Wyrównywanie (normalizacja) histogramu

Łączymy sąsiednie jasności tak, aby każda nowa jasność sumarycznie miała (możliwie najdokładniej) tyle samo pixeli.



Lokalne metody przetwarzania obrazów

Istotą metod lokalnych jest to, że:

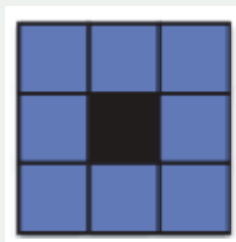
- Jasności/kolory pojedynczego pixela obrazu przetworzonego (wyjściowego) są funkcją jasności/kolorów zbioru pixeli z pewnego (ograniczonego) sąsiedztwa danego pixela.

Mówimy, że wyjściowa jasność (kolor) jest wynikiem działania *operatora o ograniczonej średnicy* (*ograniczonym promieniu, ograniczonym jądrze, ograniczonym sąsiedztwie*).

Podstawowe pytanie: Jak określamy sąsiedztwo danego pixela, czyli ilu „sąsiadów” ma każdy pixel?



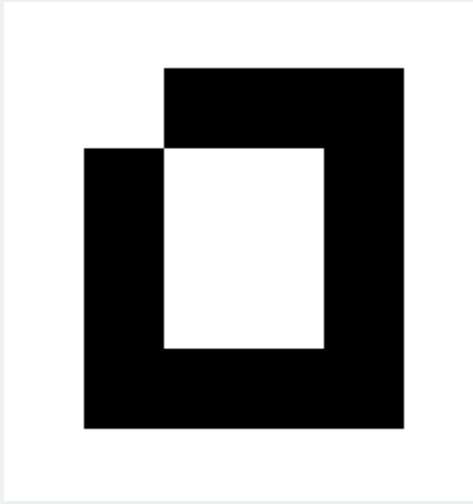
4-spójne sąsiedztwo



8-spójne sąsiedztwo

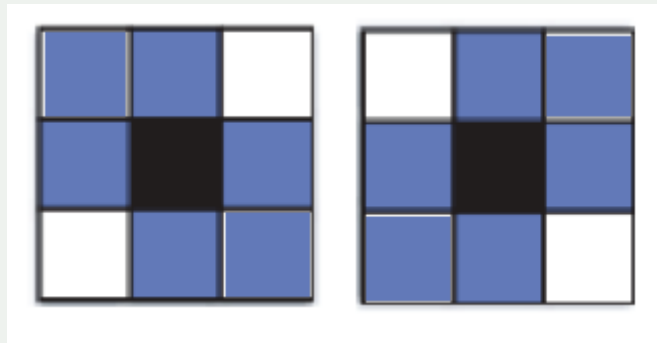
Lokalne metody przetwarzania obrazów

Problem:



Czy jest to zamknięty obszar z dziurą w środku, czy też niedomknięty obszar bez dziury w środku?

Żeby uniknąć takich dylematów, czasem (choć w praktyce bardzo rzadko) używa się pojęcia 6-spójnego sąsiedztwa (w dwóch wariantach).



Lokalne metody przetwarzania obrazów

W metodach lokalnych, jasność pixeli obrazu wyjściowego jest funkcją jasności pixeli z określonego otoczenia (sąsiedztwa) w obrazie wejściowym.

Często (choć nie zawsze) jest to funkcja w postaci liniowej kombinacji jasności pixeli z sąsiedztwa.

Przykład:

$$i_{wy}(m, n) = \sum_{k=-1}^{k=1} \sum_{l=-1}^{l=1} \frac{1}{9} i_{we}(m+k, n+l)$$

$\frac{1}{9}$	1	1	1
	1	1	1
	1	1	1

Jest to przykład ilustrujący uśrednianie (rozmywanie) obrazu.

Obraz wejściowy

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Obraz wyjściowy

Jest to przykład ilustrujący uśrednianie (rozmywanie) obrazu.

Obraz wejściowy

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Obraz wyjściowy

	0	10							

Jest to przykład ilustrujący uśrednianie (rozmywanie) obrazu.

Obraz wejściowy

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	90	0	90	90	90	0
0	0	0	90	90	90	90	90	0
0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Obraz wyjściowy

[illegible]

Jest to przykład ilustrujący uśrednianie (rozmywanie) obrazu.

Obraz wejściowy

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Obraz wyjściowy

	0	10	20	30					

Jest to przykład ilustrujący uśrednianie (rozmywanie) obrazu.

Obraz wejściowy

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Obraz wyjściowy

	0	10	20	30	30				

Jest to przykład ilustrujący uśrednianie (rozmywanie) obrazu.

Obraz wejściowy

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Obraz wyjściowy

	0	10	20	30	30				

Jest to przykład ilustrujący uśrednianie (rozmywanie) obrazu.

Obraz wejściowy

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Obraz wyjściowy

	0	10	20	30	30				
						?			
				50					

Jest to przykład ilustrujący uśrednianie (rozmywanie) obrazu.

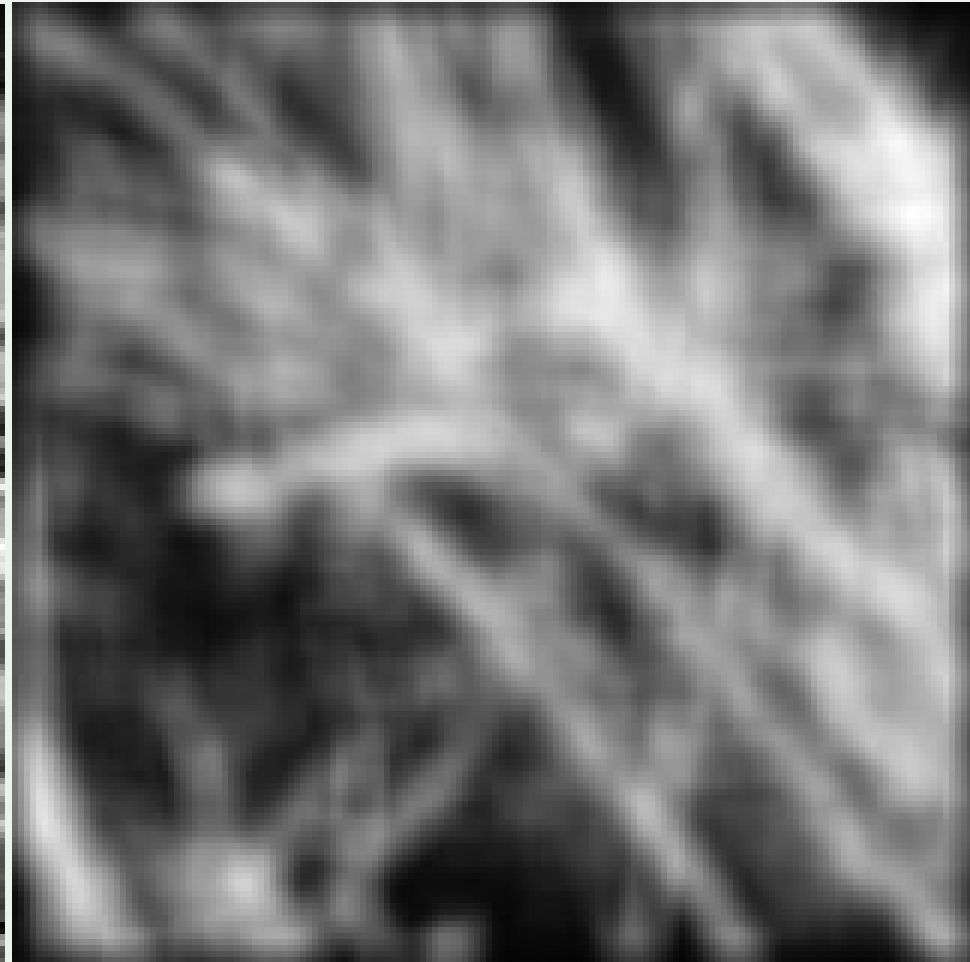
Obraz wejściowy

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Obraz wyjściowy

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

Przykład wygładzania obrazu operatorem uśredniającym.



Lokalne metody przetwarzania obrazów

Innym przykładem lokalnej metody przetwarzania obrazów jest detekcja *konturów* w obrazie.

Najprościej wykrywać punkty konturowe w obrazach czarno-białych.



Pixel konturowy (krawędziowy) w obrazie binarnym to taki, który ma co najmniej jednego sąsiada (4-spójnego lub 8-spójnego) w przeciwnym kolorze.

Formalnie można to zapisać (dla 4-spójności):

$$\begin{aligned} cont(i, j) = & [im(i, j) \oplus im(i + 1, j)] \vee [im(i, j) \oplus im(i - 1, j)] \vee \\ & [im(i, j) \oplus im(i, j + 1)] \vee [im(i, j) \oplus im(i, j - 1)] \end{aligned} \quad \oplus \text{ to operacja XOR}$$

Lokalne metody przetwarzania obrazów

Kontury w obrazach czarno-białych

Otrzymujemy w ten sposób „grube” krawędzie (punkty brzegowe białych i czarnych obszarów).

Można je „pocenić” biorąc pod uwagę tylko te pixele, które były pierwotnie białe (albo tylko czarne).

Formalnie można to zapisać:

$$cont_{\text{biały}}(i, j) = im(i, j) \wedge cont(i, j) \quad \text{"jasne" punkty konturowe}$$

albo

$$cont_{\text{czarny}}(i, j) = \overline{im(i, j)} \wedge cont(i, j) \quad \text{"ciemne" punkty konturowe}$$

Lokalne metody przetwarzania obrazów

Kontury w obrazach monochromatycznych

Nieformalnie, punkt konturowy (krawędź) to takie miejsce w obrazie, gdzie gwałtownie zmienia się jego jasność (albo kolor, dla obrazów kolorowych).

Matematycznie oznacza to, że co najmniej jedna z pochodnych cząstkowych funkcji jasności $im(x,y)$ ma dużą wartość bezwzględną:

$$\text{duża } \frac{\partial im(x,y)}{\partial x} = g_x \quad \text{pionowa krawędź}$$

lub

$$\text{duża } \frac{\partial im(x,y)}{\partial y} = g_y \quad \text{pozioma krawędź}$$

Inaczej mówiąc, możemy utworzyć nową wersję obrazu, taką, ze:

$$gi(x,y) = \left| \frac{\partial im(x,y)}{\partial x} \right| + \left| \frac{\partial im(x,y)}{\partial y} \right| = |g_x| + |g_y|$$

albo

$$gi(x,y) = \sqrt{\left(\frac{\partial im(x,y)}{\partial x} \right)^2 + \left(\frac{\partial im(x,y)}{\partial y} \right)^2} = \sqrt{(g_x)^2 + (g_y)^2}$$

Otrzymane w ten sposób obrazy nazywamy **obrazami gradientowymi**.

Lokalne metody przetwarzania obrazów

Miarą intensywności konturu („wartość konturowości”) jest $g = \sqrt{g_x^2 + g_y^2}$ albo $g = |g_x| + |g_y|$.

Często jednak używamy po prostu arbitralnie wybranego progu, żeby odróżnić punkty konturowe (intensywność konturu powyżej progu) od innych.

W ten sposób tworzymy binarne obrazy konturowe.



Lokalne metody przetwarzania obrazów

Kontury w obrazach monochromatycznych

W obrazach cyfrowych używamy dyskretnych aproksymacji pochodnych cząstkowych, np.

$$\frac{\partial im(x, y)}{\partial x} \approx im(x, y) - im(x - 1, y)$$

oraz

$$\frac{\partial im(x, y)}{\partial y} \approx im(x, y) - im(x, y - 1)$$

ale częściej używamy bardziej skomplikowanych aproksymacji pochodnych cząstkowych, które dodatkowo lekko uśredniają obraz, żeby nie wykrywać krawędzi wynikających z drobnych zmian jasności.

Jedną z popularnych aproksymacji są tzw. *operatory Sobela*:

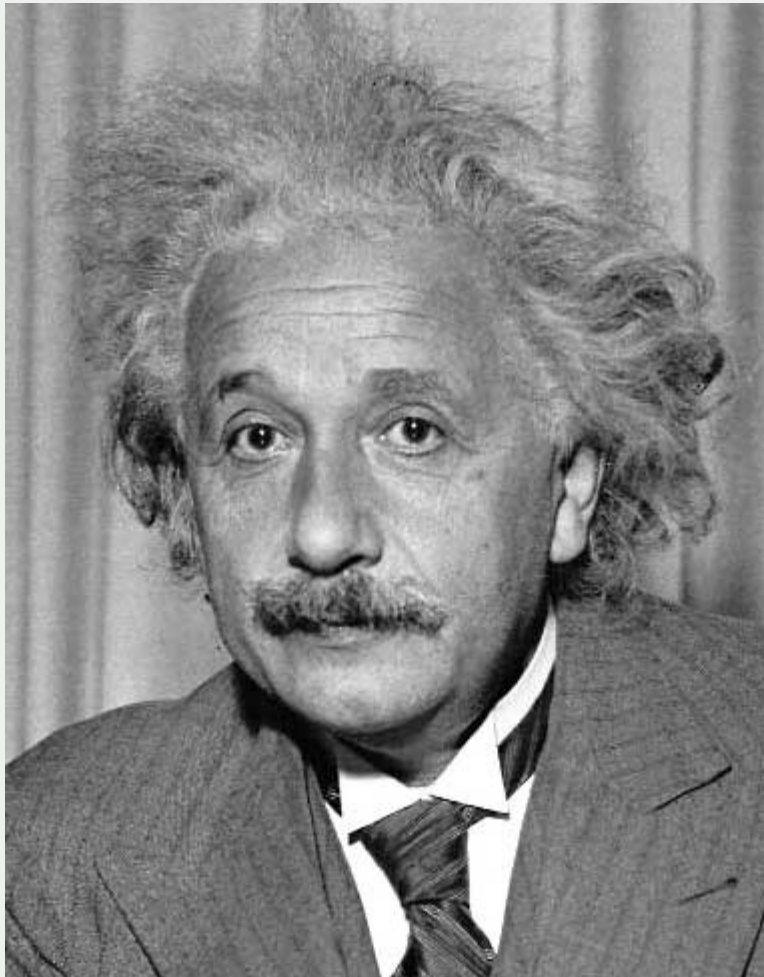
$$G_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} \quad \text{oraz} \quad G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

$$\frac{\partial im(x, y)}{\partial x} \approx 2im(x - 1, y) + im(x - 1, y - 1) + im(x - 1, y + 1) - 2im(x + 1, y) - im(x + 1, y - 1) + im(x + 1, y + 1)$$

$$\frac{\partial im(x, y)}{\partial y} \approx \text{podobnie}$$

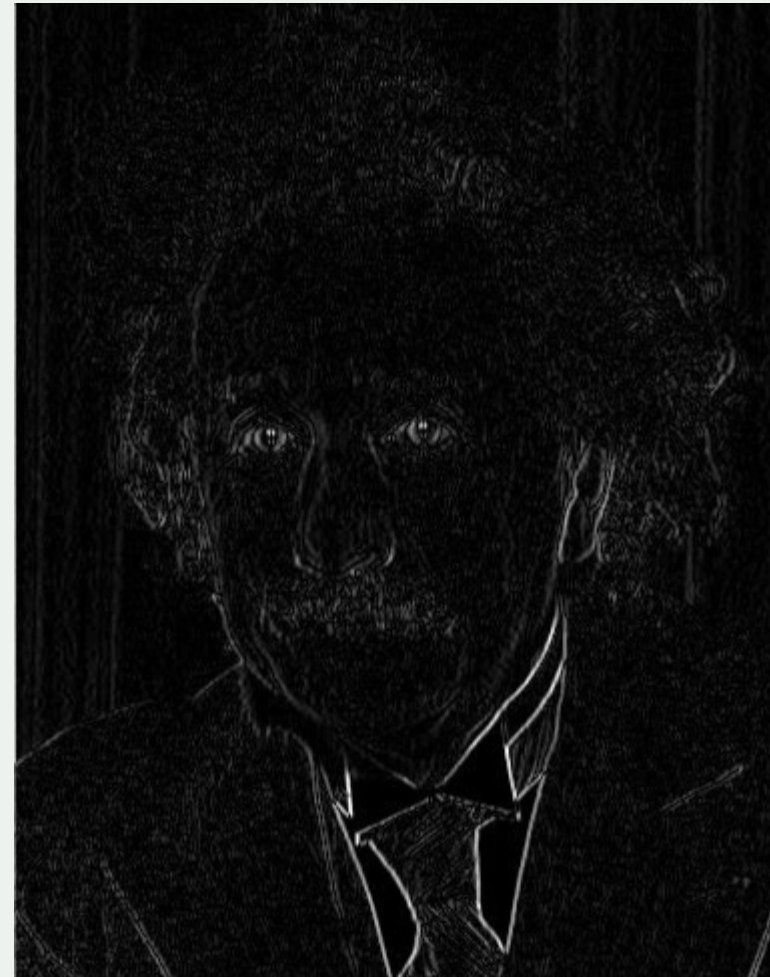
Lokalne metody przetwarzania obrazów

wykrywanie krawędzi pionowych (operatory Sobela)



1	0	-1
2	0	-2
1	0	-1

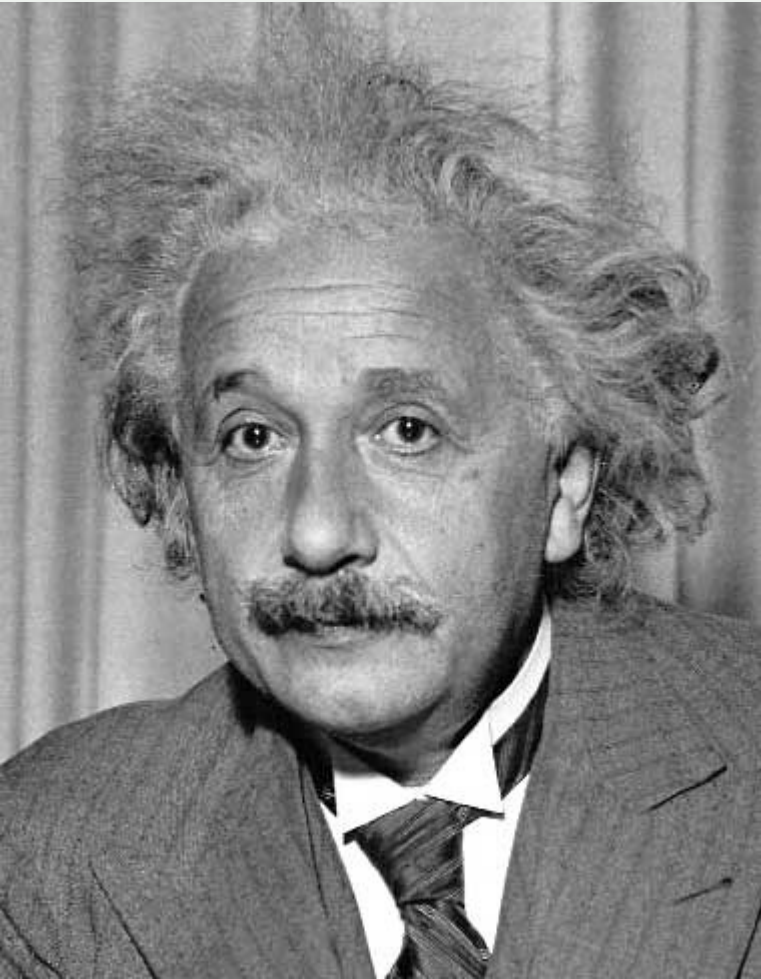
Sobel



Wyświetlana jest wartość absolutna operatora ***G***

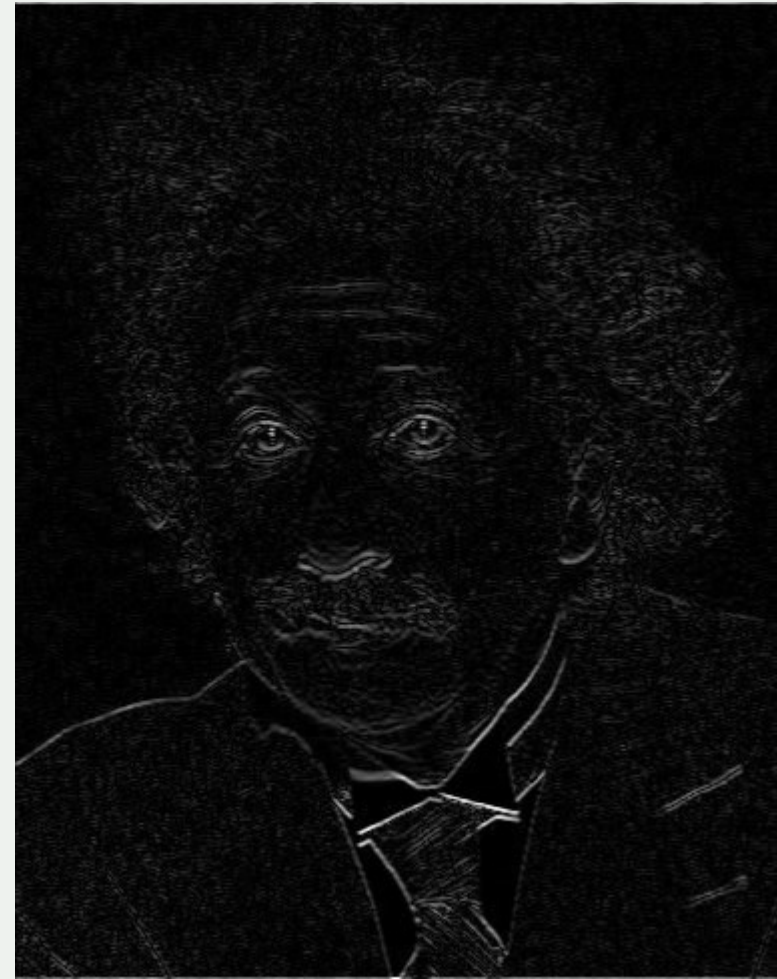
Lokalne metody przetwarzania obrazów

wykrywanie krawędzi poziomych (operatory Sobela)



1	2	1
0	0	0
-1	-2	-1

Sobel



Wyświetlana jest wartość absolutna operatora ***G***

Lokalne metody przetwarzania obrazów

Istnieją też inne operatory do wykrywania krawędzi w obrazach monochromatycznych, np.

operatory Prewitta

$$\begin{bmatrix} +1 & 0 & -1 \\ +1 & 0 & -1 \\ +1 & 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} +1 & +1 & +1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

operatory Robertsa

+1	0
0	-1

0	+1
-1	0

Popularny jest również tzw. *detektor Canny'ego*, które opiera się na bardziej skomplikowanym modelu matematycznym.

Lokalne metody przetwarzania obrazów

Binaryzację obrazów można również realizować metodą lokalną (jest to bardziej ogólny sposób).

Używamy tzw. **progowania adaptacyjnego**, gdzie próg jest wyznaczany lokalnie (na podstawie jasności w ograniczonym otoczeniu danego pixela).

Przykład

What Is Image Filtering in the Spatial Domain?

Filtering is a technique for modifying or enhancing an image. For example, you can filter an image to emphasize certain features or remove other features. Image processing operations implemented with filtering include smoothing, sharpening, and edge enhancement.

Filtering is a *neighborhood operation*, in which the value of any given pixel in the output image is determined by applying some algorithm to the values of the pixels in the neighborhood of the corresponding input pixel. A pixel's neighborhood is some set of pixels, defined by their locations relative to that pixel. (See Neighborhood or Block Processing: An Overview for a general discussion of neighborhood operations.) *Linear filtering* is filtering in which the value of an output pixel is a linear combination of the values of the pixels in the input pixel's neighborhood.

Convolution

Linear filtering of an image is accomplished through an operation called *convolution*. Convolution is a neighborhood operation in which each output pixel is the weighted sum of neighboring input pixels. The matrix of weights is called the *convolution kernel*, also known as the *filter*. A convolution kernel is a correlation kernel that has been rotated 180 degrees.

For example, suppose the image is

```
A = [ 17  24  1  8 15
      23  5  7 14 16
        4  6 13 28 22
       18 12 19 21  3
```

What Is Image Filtering in the Spatial Domain?

Filtering is a technique for modifying or enhancing an image. For example, you can filter an image to emphasize certain features or remove other features. Image processing operations implemented with filtering include smoothing, sharpening, and edge enhancement.

Filtering is a *neighborhood operation*, in which the value of any given pixel in the output image is determined by applying some algorithm to the values of the pixels in the neighborhood of the corresponding input pixel. A pixel's neighborhood is some set of pixels, defined by their locations relative to that pixel. (See Neighborhood or Block Processing: An Overview for a general discussion of neighborhood operations.) *Linear filtering* is filtering in which the value of an output pixel is a linear combination of the values of the pixels in the input pixel's neighborhood.

Convolution

Linear filtering of an image is accomplished through an operation called *convolution*. Convolution is a neighborhood operation in which each output pixel is the weighted sum of neighboring input pixels. The matrix of weights is called the *convolution kernel*, also known as the *filter*. A convolution kernel is a correlation kernel that has been rotated 180 degrees.

For example, suppose the image is

```
A = [ 17  24  1  8 15
      23  5  7 14 16
        4  6 13 28 22
       18 12 19 21  3
```

Globalne metody przetwarzania obrazów

Istotą metod globalnych jest to, że:

- Należy wykonać obliczenia wymagające użycia całego obrazu, aby wyznaczyć nowe jasności (kolory) poszczególnych pixeli.

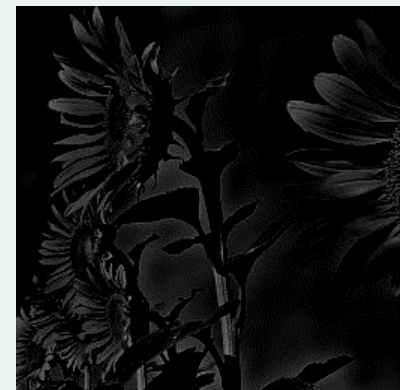
Mówimy, że wyjściowa jasność (kolor) pixela jest wynikiem działania *operatora o nieograniczonej średnicy*.

Najbardziej typowym przykładem takich metod są techniki przetwarzania obrazu z użyciem transformat (np. transformaty Fouriera, cosinusowe, falkowe, itp.).

Najpierw obliczamy *transformatę całego obrazu*, następnie dokonujemy wybranych działań na transformacie, a na końcu generujemy obraz wyjściowy poprzez obliczenie *odwrotnej transformaty*.



$\xRightarrow{FFT}$ transformata
(modyfikowana) $\xRightarrow{IFFT}$



Globalne metody przetwarzania obrazów

Można jednak zauważyć, że na pozór punktowa binaryzacja obrazów, ale z progiem wyznaczonym metodą Otsu, też powinna być uznana za metodę globalną (co do ogólnej zasady).

Ponieważ próg wyznaczamy na podstawie histogramu (obliczanego dla całego obrazu) może się zdarzyć, że zmiana jasności pojedynczych pixeli tak zmodyfikuje próg binaryzacji, że wiele pixeli w całym obrazie zmieni się z białych na czarne (lub na odwrót) 😊.

Lokalne/globalne metody przetwarzania obrazów

UWAGI

1. *Rozmiar operatora* to wymiar tablicy (macierzy) określającej, ile sąsiednich pixeli (i które z nich) używamy do obliczenia wartości operacji dla danego pixela (np. rozmiar 3x3, 11x5, itp.).
Alternatywnie można podawać *średnicę operatora* używanego do przetwarzania obrazu, czyli maksymalną odległość pomiędzy pixelami używanymi do obliczania wyniku danej operacji.
2. Średnica operatora lokalnego to oczywiście ZERO, zaś średnica operatorów globalnych jest nieokreślona (i zależna od rozdzielczości/rozmiarów danego obrazu).

Rozmiar operatora ma duże praktyczne znaczenie, gdyż determinuje możliwość zrównoleglenia operacji (lub łatwość ich implementacji sprzętowej).

Lokalne/globalne metody przetwarzania obrazów

UWAGI

Operatory Sobela (służące do wyznaczania krawędzi w obrazach cyfrowych) opisywane są następującymi macierzami:

1	0	-1
2	0	-2
1	0	-1

Sobel (x)

1	2	1
0	0	0
-1	-2	-1

Sobel (y)

Są to więc operatory o średnicy **$2 \cdot \sqrt{2}$** (albo **2** jeśli używamy metryki **L_∞** , albo **4** jeśli używamy metryki **L_1**).

Ale rozmiar tych operatorów, to zawsze **3x3**.

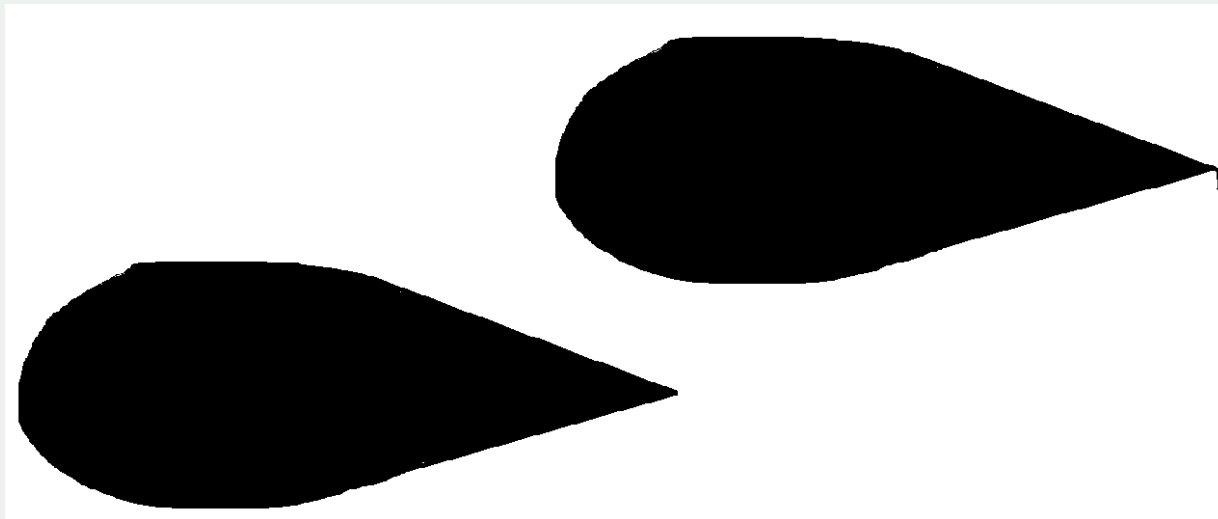
Lokalne/globalne metody przetwarzania obrazów

PRZYKŁADY

Czy wyznaczanie wypukłości/wklęsłości obszarów jest operacją punktową, lokalną, czy globalną?

Obszar jest wypukły, jeśli odcinek łączący dwa dowolne punkty tego obszaru w całości zawiera się w danym obszarze.

A więc musimy wykonywać działania o nieokreślenie dużej średnicy (maksymalna odległość między punktami należącymi do tego samego obszaru).



Lokalne/globalne metody przetwarzania obrazów

PRZYKŁADY

Czy znajdowanie liczby spójnych obiektów pierwszego planu (zakładamy, że nie mają one „dziur”) w obrazach binarnych jest operacją punktową, lokalną, czy globalną?



Pozornie wydaje się to operacją globalną (spójność oznacza, że można poprowadzić ciągłą linię między dowolnymi pixelami danego obszaru).

W rzeczywistości jest to operacja lokalna o bardzo małym operatorze.

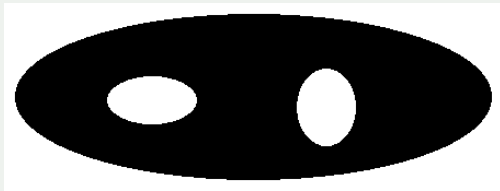
Przy założeniu, że obszary nie mają „dziur”, wystarczy obliczyć tzw. *liczbę Eulera* obrazu.

Lokalne/globalne metody przetwarzania obrazów

PRZYKŁADY

Liczba Eulera obrazu to liczba spójnych składowych minus liczba „dziur” w tych spójnych składowych:

$$E = C - H$$



$$E = 1 - 2 = -1$$



$$E = 2 - 1 = 1$$



$$E = 3 - 5 = -2$$

Jeżeli obszary nie mają „dziur”, liczba spójnych obszarów w obrazie równa się zatem po prostu *liczbie Eulera* tego obrazu:

$$E = C - 0 = C$$

Lokalne/globalne metody przetwarzania obrazów

UWAGI OGÓLNE I SZCZEGÓLNE

Okazuje się, *liczbę Eulera* obrazów cyfrowych, w których używamy pojęcia 4-spójności dla określania spójnych obszarów pierwszego planu (obiektów) można obliczyć przy użyciu operacji lokalnych o rozmiarach nie większych niż 2x2.

$$E = S1 - S2 + S3$$

Gdzie

S1: liczba pixeli pierwszego planu (obiektów)

S2: liczba spójnych par pixeli pierwszego planu (obiektów)

S3: liczba spójnych „czwórek” pierwszego planu (obiektów).

